WEEK 10

알고리즘 심화

알고리즘 심화

사용자 정의 함수와 알고리즘 설계

복잡한 문제를 해결하기 위해 **나만의 함수**를 정의하여 코드를 구조화하고, 데이터를 효율적으로 분석하는 **정렬 및 평가 알고리즘**을 구현합니다.

함수 구조화 (Functions)

- **def, return** 및 매개변수 활용
- 절댓값(my_abs), 짝수 판별(is_even)
- 성적 등급 산출(grade) 함수 구현

정렬 알고리즘 (Sorting)

- **버블 정렬(Bubble Sort)** 원리 이해
- 데이터의 최댓값, 최솟값 찾기
- 전체 평균 계산 알고리즘 응용

종합 실습 (Evaluation)

- **과제 자동 평가 시스템** 구축
- 제출 시간, 주석, 감점 요인 분석
- 조건에 따른 최종 점수 산출 로직

함수의 개념과 구조

함수는 여러 줄의 코드를 하나의 이름으로 묶어 필요할 때마다 재사용할 수 있게 해주는 프로그래밍의 핵심 요소입니다.

함수의 3요소

1. **def**: 함수를 정의(define)함을 알리는 키워드입니다.
2. **매개변수**: 함수 내부로 데이터를 전달하는 입력 통로입니다.
3. **return**: 함수 실행 결과를 호출한 곳으로 반환합니다.

재사용성 & 모듈화

코드를 기능 단위로 분리하여 중복을 줄이고, 가독성과 유지보수성을 높입니다. 한 번 정의하면 어디서든 호출 가능합니다.

입력과 출력

함수는 **입력(매개변수)**을 받아 내부 로직을 처리한 후, **출력(반환값)**을 돌려주는 블랙박스와 같습니다.

```
# 함수 정의 (Definition)
def function_name(param1, param2):
    result = param1 + param2 # 처리
    return result # 반환

# 함수 호출 (Call)
value = function_name(10, 20)
```

나만의 알고리즘: my_abs, is_even

절댓값 함수: my_abs(x)

```
if x >= 0: return x
```

정수 x를 입력받아 절댓값을 반환합니다. x가 0보다 크거나 같으면 그대로 x를, 0보다 작으면 -x를 반환하여 항상 양수 값을 얻습니다.

짝수 판별: is_even(n)

```
return n % 2 == 0
```

정수 n을 입력받아 짝수 여부를 판단합니다. 2로 나눈 나머지가 0이면 True(짝수), 그렇지 않으면 False(홀수)를 반환합니다.

명세 기반 구현

```
Input / Output
```

함수 설계 시 입력(매개변수)의 타입과 출력(반환값)의 형식을 명확히 정의합니다. 경계값(예: 0) 테스트를 통해 로직의 정확성을 검증합니다.

```
def my_abs(x): return x if x >= 0 else -x
def is_even(n): return True if n % 2 == 0 else False
```

성적 등급 함수 grade(score)

점수에 따른 학점 변환 로직

사용자가 입력한 0~100점 사이의 점수(score)를 입력받아 if-elif-else 조건문을 통해 평가하고, 해당하는 등급(A~F)을 반환하는 함수를 구현합니다.

90점 이상 → A
80점 이상 → B

70점 이상 → C
60점 이상 → D

그 외 (60점 미만) → F 반환

버블 정렬 알고리즘 (Bubble Sort)

알고리즘 심화 (p.128)

인접한 두 요소를 비교하여 순서가 맞지 않으면 서로 교환(Swap)하는 방식의 정렬 알고리즘입니다.

한 번의 반복(Pass)이 끝날 때마다 가장 큰 값이 리스트의 맨 뒤로 이동하는 특징이 있어 마치 거품이 수면으로 올라오는 것과 같습니다.

01

인접 요소 비교 (Comparison)

```
if a[j] > a[j+1]:
```

02

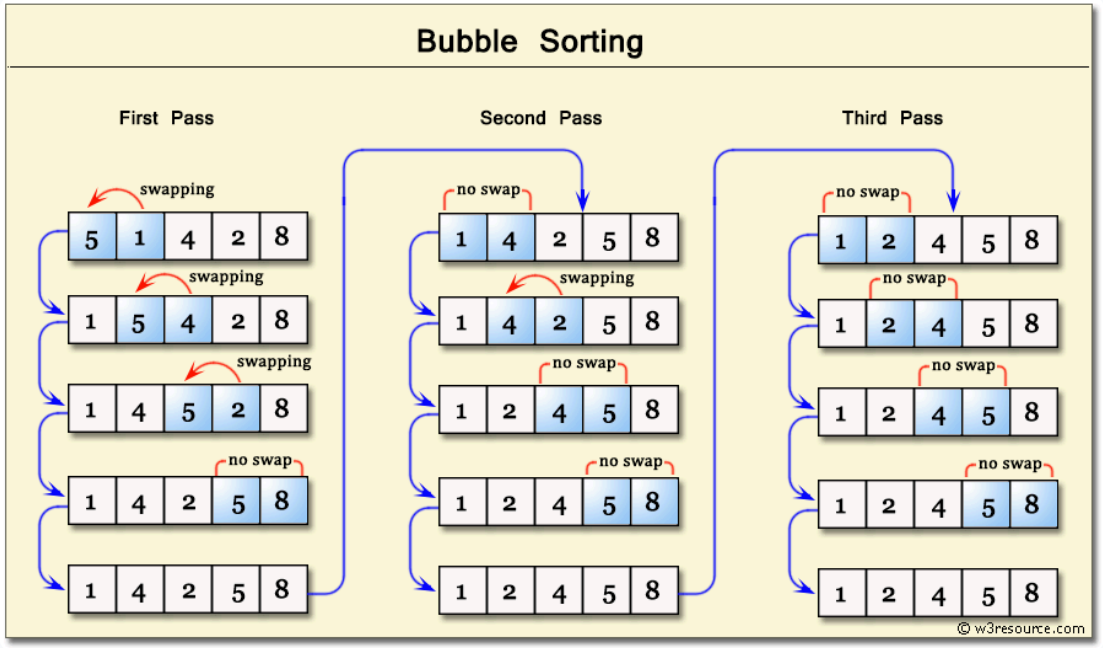
위치 교환 (Swap)

```
temp = a[j]; a[j]=a[j+1];
a[j+1]=temp
```

03

반복 수행 (Iteration)

큰 값이 맨 뒤로 이동하며 범위 축소



버블 정렬의 단계별 시각화 (인접 요소 비교 및 교환)

버블 정렬 응용: 최댓값·최솟값·평균

정렬이 완료된 리스트를 활용하면 최솟값, 최댓값, 평균과 같은 통계적 지표를 매우 쉽게 계산할 수 있습니다.

핵심 포인트

1. **최솟값**: 오름차순 정렬 리스트의 **첫 번째 요소 (a[0])**
2. **최댓값**: 오름차순 정렬 리스트의 **마지막 요소 (a[-1])**
3. **평균**: 전체 요소의 합(sum)을 개수(len)로 나눈 값

데이터 분석

정렬된 데이터는 인덱스만으로 극단값(Min/Max)을 즉시 찾을 수 있어 효율적입니다. 평균값은 전체 데이터의 중심 경향성을 보여줍니다.

```
# a는 이미 오름차순 정렬된 리스트라고 가정
min_val = a[0] # 최솟값
max_val = a[-1] # 최댓값
avg_val = sum(a) / len(a) # 평균값

print(f"최소: {min_val}, 최대: {max_val}, 평균: {avg_val:.2f}")
```

종합 실습: 과제 자동 평가

submit_assignment() 함수 구현

제출 상황에 따른 4가지 정보를 입력받아 최종 점수를 계산하고, 이에 따른 학점 등급(A~F)을 반환하는 자동화 알고리즘을 함수로 구현해 봅니다.

```
def submit_assignment(on_time, late_min,
has_comment, penalty):
```

01
4개 매개변수
(Input)

02
감점 계산 로직
(Process)

03
최종 등급 반환
(Return)

10주차
요약 및 예고

오늘 우리는 사용자 정의 함수의 구조(def/return)를 익히고, my_abs, is_even, grade 함수를 구현했습니다. 또한 버블 정렬 알고리즘을 이해하고, 이를 활용해 과제 자동 평가 시스템을 설계했습니다.

핵심 포인트 & 다음 주차:

01
함수 활용:
정의, 매개변수,
반환

02
알고리즘:
버블 정렬, 통계
계산

NEXT
에러와 디버깅:
Thonny 활용법